

LAB 3

ARCHITECTURE SELECTION AND COMPONENT DIAGRAM

Food Bank Surplus Redistribution Platform

NAME :SUNIL A. SHINDOLKAR

SRN : PES1UG25CS848

SEC : L

1. Problem Context

The Food Bank Surplus Redistribution Platform is a food-rescue system in which restaurants and grocery stores post surplus edible food batches, while verified local shelters reserve suitable batches before their expiry. The two main stakeholders shown in the component diagram are the Donor Partner and the Shelter Coordinator. The platform must support food-batch posting, reservation, inventory and expiry tracking, notifications to nearby shelters, and payment processing.

2. Selected Architecture

Microservices Architecture is selected for this platform. The system is divided into independent services/components that communicate through clearly defined APIs. The diagram represents five main components: Donor Interface, Order Manager Component, Food Inventory & Expiry, Notification Service, and Payment Service Component.

3. Component Responsibilities

Component	Responsibility
Donor Interface	Provides the interface through which a donor partner posts surplus food batches and expiry information.
Order Manager Component	Coordinates food-batch reservations and communicates with inventory, notification, and payment services.
Food Inventory & Expiry	Tracks food quantity/availability and expiry information so expired batches are not treated as active.
Notification Service	Handles notifications to shelters, including alerts for nearby shelters when high-quantity surplus food is listed.
Payment Service Component	Provides payment-processing functionality through the Payment Processing API.

4. Interface and Interaction Design

The component diagram uses UML provided/required interface notation. A ball represents a Provided Interface, meaning that the connected component offers the service. A socket represents a Required Interface, meaning that the connected component needs the service. The main interfaces in the diagram are Food Batch API, Inventory / Expiry API, Notification API, and Payment Processing API. The Shelter Coordinator also interacts with the Order Manager through the Reservation API.

The Donor Interface provides the Food Batch API to the Order Manager. The Order Manager requires the Inventory / Expiry API from Food Inventory & Expiry, the Notification API from Notification Service, and the Payment Processing API from Payment Service. This separation keeps service responsibilities clear and makes the communication paths explicit.

5. Why Microservices Architecture?

First, the architecture provides independent services. Food inventory, order management, notifications, and payment processing can be developed and maintained as separate components. A change to one service does not require the entire platform to be redesigned.

Second, the architecture supports scalability and fast notification handling. The problem statement requires instant push notifications to shelters within a 5 km radius when high-quantity surplus food is listed. The Notification Service can handle notification workload independently and can be scaled when the number of users or notifications increases.

6. Security Considerations

Security can be improved by separating access through service-specific APIs. Each service can enforce appropriate authentication and authorization rules, limiting which users or components can access particular functions. Separation also reduces the impact of an issue in one service because other services remain logically isolated.

7. Performance Considerations

Microservices allow individual services to be scaled according to demand. For example, if many shelters must receive notifications at the same time, the Notification Service can be scaled without scaling every other component. Similarly, inventory and order-management workloads can be handled independently. This supports responsive operation and helps the system meet the required notification performance.

8. Acceptance and Expected Behaviour

A successful implementation should allow a shelter to reserve an available food batch and receive pickup verification, while expired food batches must not remain active. For the non-functional requirement, benchmarking should be used to confirm the target notification latency and security standards under simulated peak load.

9. Conclusion

The proposed Microservices Architecture is suitable for the Food Bank Surplus Redistribution Platform because it separates the major functions into clear components, defines their interactions through APIs, supports independent scaling, and provides a practical structure for security and performance controls. The accompanying component diagram shows the two stakeholders, five main components, and the required service interfaces.